Part 1 — Goal and Vulnerability Summary

This lesson demonstrates an insecure cloud configuration in the DVSA feedback file upload workflow. The affected components are the public S3 feedback bucket and the Lambda function DVSA-FEEDBACK-UPLOADS, which is automatically triggered when new objects are uploaded. The security impact is that an attacker can upload arbitrary files directly into the bucket and cause backend processing on attacker-controlled object names. Because the backend uses unsafe shell execution with insufficient filename validation, this increases the risk of command injection or unintended backend behavior.

Part 2 — Why This Works / Root Cause

The vulnerability is possible because the S3 feedback bucket is misconfigured to allow public object operations. Public access blocking is disabled, and the bucket policy allows Principal: "*" to perform actions such as s3:PutObject, s3:GetObject, and s3:DeleteObject. In addition, every uploaded object triggers the Lambda function DVSA-FEEDBACK-UPLOADS. Inside that function, the uploaded filename is passed into os.system(...), while the is_safe() check always returns True. As a result, attacker-controlled filenames can reach unsafe backend command processing.

Part 3) Environment and Setup

The testing was performed on my own DVSA deployment in AWS in the us-east-1 region. The main components involved in this lesson were the S3 bucket dvsa-feedback-bucket-716522590330-us-east-1 and the Lambda function DVSA-FEEDBACK-UPLOADS. The bucket was configured with an event notification for **All object create events**, which invoked the Lambda function whenever a new object was uploaded. The AWS services used during the investigation were Amazon S3, AWS Lambda, and Amazon CloudWatch Logs. The tools used were the AWS Console and the DVSA deployment already prepared for the course project.

Part 4) Reproduction Steps

1. Opened the AWS Console and navigated to the S3 bucket dvsa-feedback-bucket-716522590330-us-east-1.
2. Reviewed the bucket properties and confirmed that it had an event notification configured for **All object create events** with destination Lambda function DVSA-FEEDBACK-UPLOADS.
3. Reviewed the bucket permissions and confirmed that **Block Public Access** was disabled.
4. Examined the bucket policy and observed that it allowed Principal: "*" to perform public object actions including s3:PutObject, s3:GetObject, and s3:DeleteObject.
5. Uploaded a harmless test file named lesson4-test.txt into the feedback bucket using the AWS Console.
6. Confirmed that the file appeared successfully in the bucket object list.
7. Opened Amazon CloudWatch Logs and navigated to the log group for /aws/lambda/DVSA-FEEDBACK-UPLOADS.
8. Verified that a new Lambda invocation occurred immediately after the upload and that the log event showed ObjectCreated:Put for the uploaded object key lesson4-test.txt.
9. Uploaded a second file with a special filename, lesson4;test.txt, to test how the backend handled attacker-controlled object names.
10. Opened the newest CloudWatch log stream and confirmed that the uploaded key appeared in the event record in URL-encoded form as lesson4%3Btest.txt.
11. Observed shell error messages in the Lambda logs, including sh: test.txt: command not found and sh: test.txt.txt: command not found, showing that the uploaded filename reached unsafe shell command processing.

Part 5) Evidence and Proof

The vulnerability was confirmed using S3 configuration evidence, successful upload evidence, and CloudWatch log evidence. First, the

feedback bucket permissions showed that Block Public Access was disabled and that the bucket policy granted broad object access to Principal: "*". Second, the bucket properties showed that all object create events triggered the Lambda function DVSA-FEEDBACK-UPLOADS. Third, I successfully uploaded a harmless test file and confirmed that it triggered a Lambda invocation. Finally, I uploaded a file with a special filename and observed shell-related errors in CloudWatch Logs, showing that attacker-controlled filenames reached unsafe backend processing.

Bucket policy Edit

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. Learn more


```
{
  "Sid": "PublicWritefeedbackStatement",
  "Effect": "Allow",
  "Principal": "*",
  "Action": [
    "s3:PutObject",
    "s3:PutObjectAcl",
    "s3:GetObject",
    "s3:DeleteObject"
  ],
  "Resource": "arn:aws:s3:::dvsa-feedback-bucket-716522590330-us-east-1/*"
},
{
  "Sid": "AdminListfeedbackStatement",
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::716522590330:root"
  },
  "Action": "s3:*",
  "Resource": "arn:aws:s3:::dvsa-feedback-bucket-716522590330-us-east-1"
```


Figure 1: Feedback bucket permissions showing public access disabled protections and a bucket policy granting broad object access to Principal: "".*

Event notifications (1) Edit Delete Create event notification

Send a notification when specific events occur in your bucket. Learn more

Name	Event types	Filters	Destination type	Destination
Y2VkNWMzYTctNmY5NC00ZGZkLWIzYjUtYjU4NWRiN2RmMzYy	All object create events	-	Lambda function	DVSA-FEEDBACK-UPLOADS

Figure 2: S3 event notification showing that every new uploaded object triggers the Lambda function DVSA-FEEDBACK-UPLOADS.

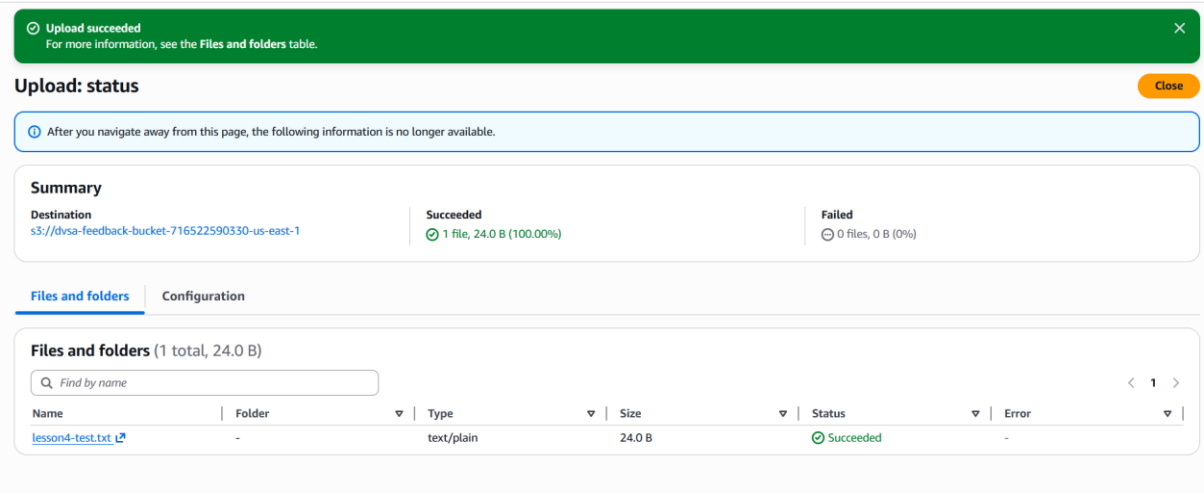


Figure 3: Successful upload of a harmless test file (lesson4-test.txt) into the feedback bucket.

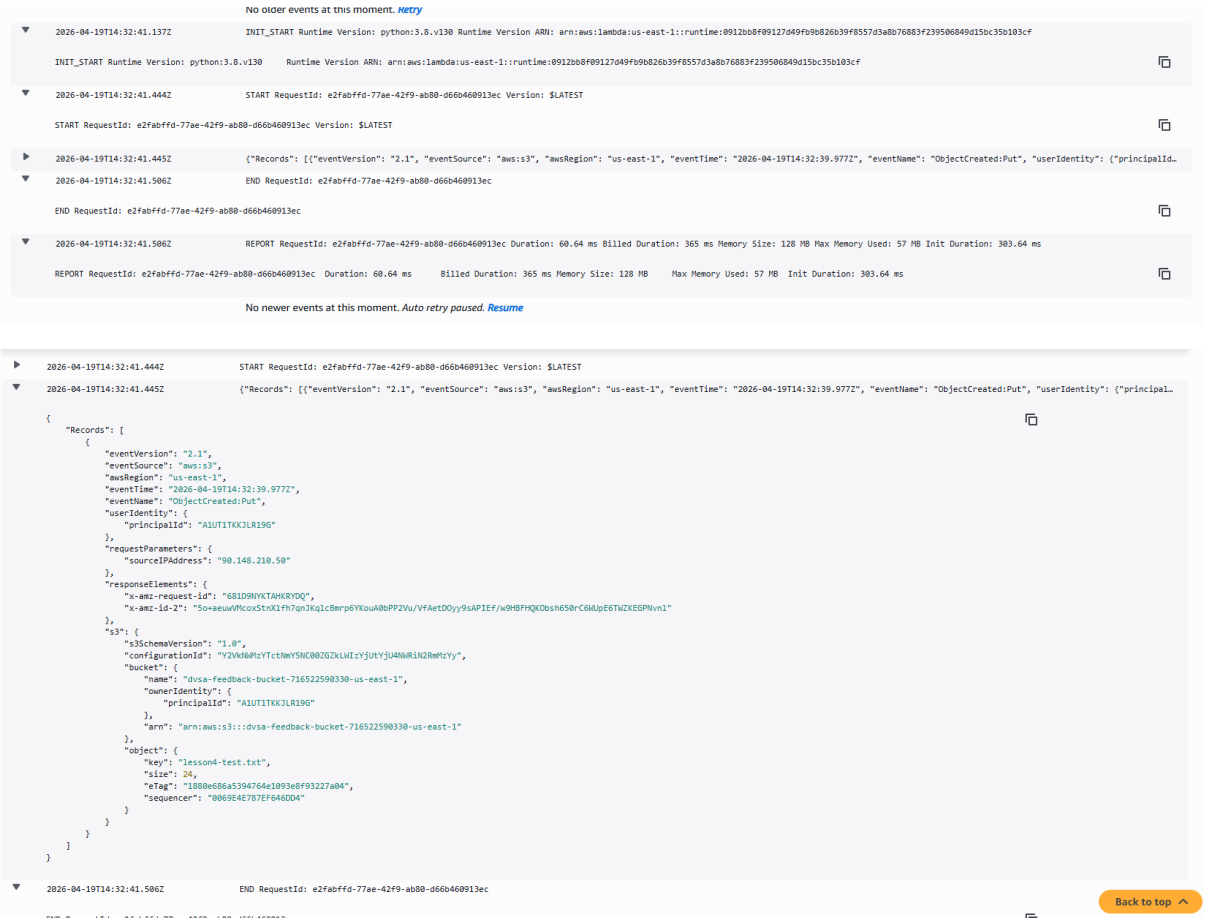


Figure 4: CloudWatch Logs showing that uploading lesson4-test.txt triggered the Lambda through an S3 ObjectCreated:Put event.

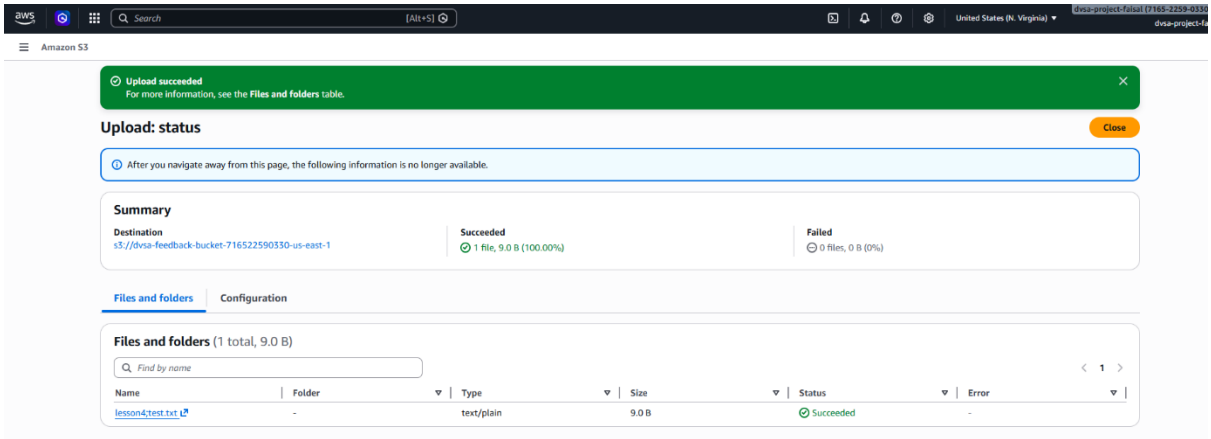


Figure 5: Uploaded file with attacker-controlled special characters in the filename (lesson4;test.txt).

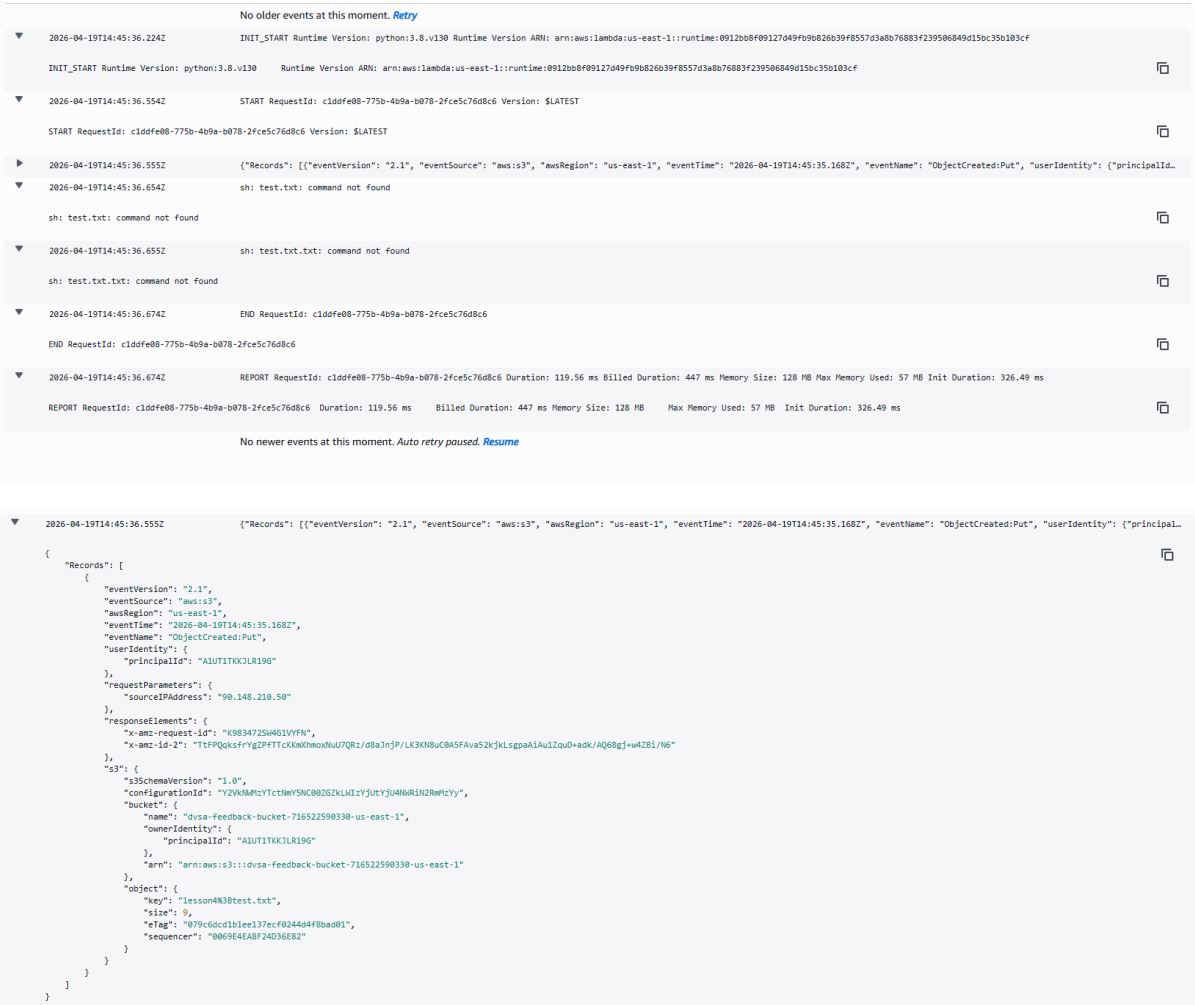


Figure 6: CloudWatch Logs showing unsafe shell-side processing of the uploaded special filename, which appeared in URL-encoded form as lesson4%3Btest.txt.

Together, these results prove both the insecure cloud configuration and the unsafe backend processing path.

Part 6) Fix Strategy / Probable Mitigation

The vulnerability should be fixed in two places. First, the S3 feedback bucket must be hardened by enabling Block Public Access and removing any bucket policy statements that allow public object access through Principal: "*". Public users should not be allowed to upload, read, or delete objects directly in the bucket.

Second, the backend Lambda function DVSA-FEEDBACK-UPLOADS must stop using unsafe shell execution with attacker-controlled filenames. The code should avoid os.system(...) entirely and instead use safe file-handling functions such as Python file operations or subprocess. Run(...) with strict argument handling if external commands are absolutely necessary. In addition, uploaded filenames should be validated using a strict allowlist so that only expected characters and safe file extensions are accepted.

Together, these changes would remove both the insecure cloud configuration and the unsafe backend processing path.

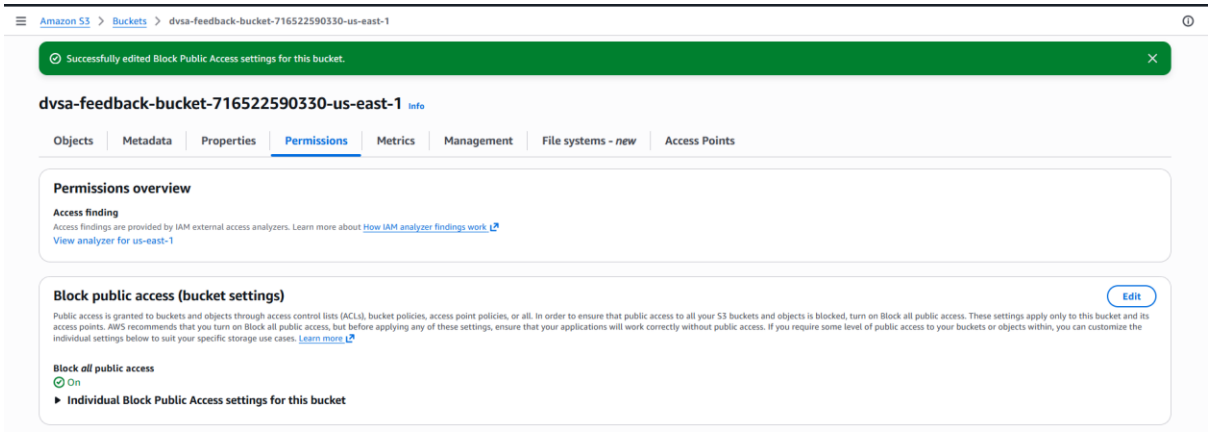
Part 7) Code / Config Changes

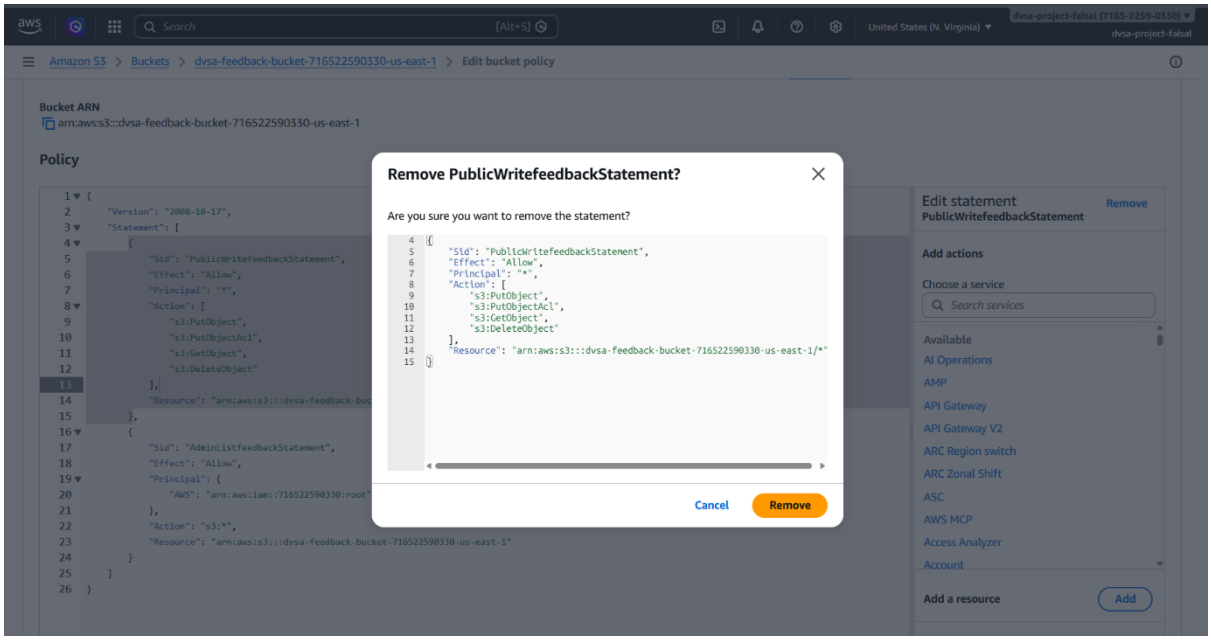
The first remediation was applied in the S3 configuration of the feedback bucket dvsa-feedback-bucket-716522590330-us-east-1. In the **Permissions** tab, **Block Public Access** was changed from **Off** to **On**.

In addition, the public bucket policy statement with Sid: "PublicWritefeedbackStatement" was removed. Before the fix, that statement allowed Principal: "*" to perform actions such as s3:PutObject, s3:PutObjectAcl, s3:GetObject, and s3:DeleteObject on objects in the feedback bucket. After the fix, only the administrative statement for the AWS account remained.

These changes removed the public anonymous object access that allowed untrusted users to upload and manage files directly in the feedback bucket.

The Lambda event notification was not removed because the intended workflow still required backend processing of feedback uploads; only the public exposure of the bucket was corrected.





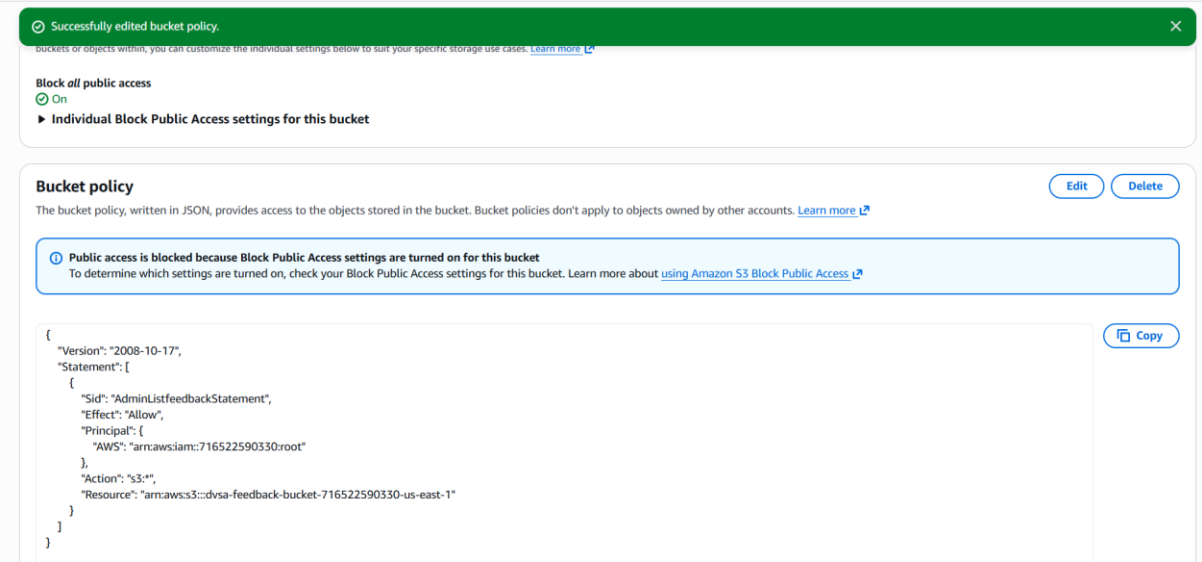
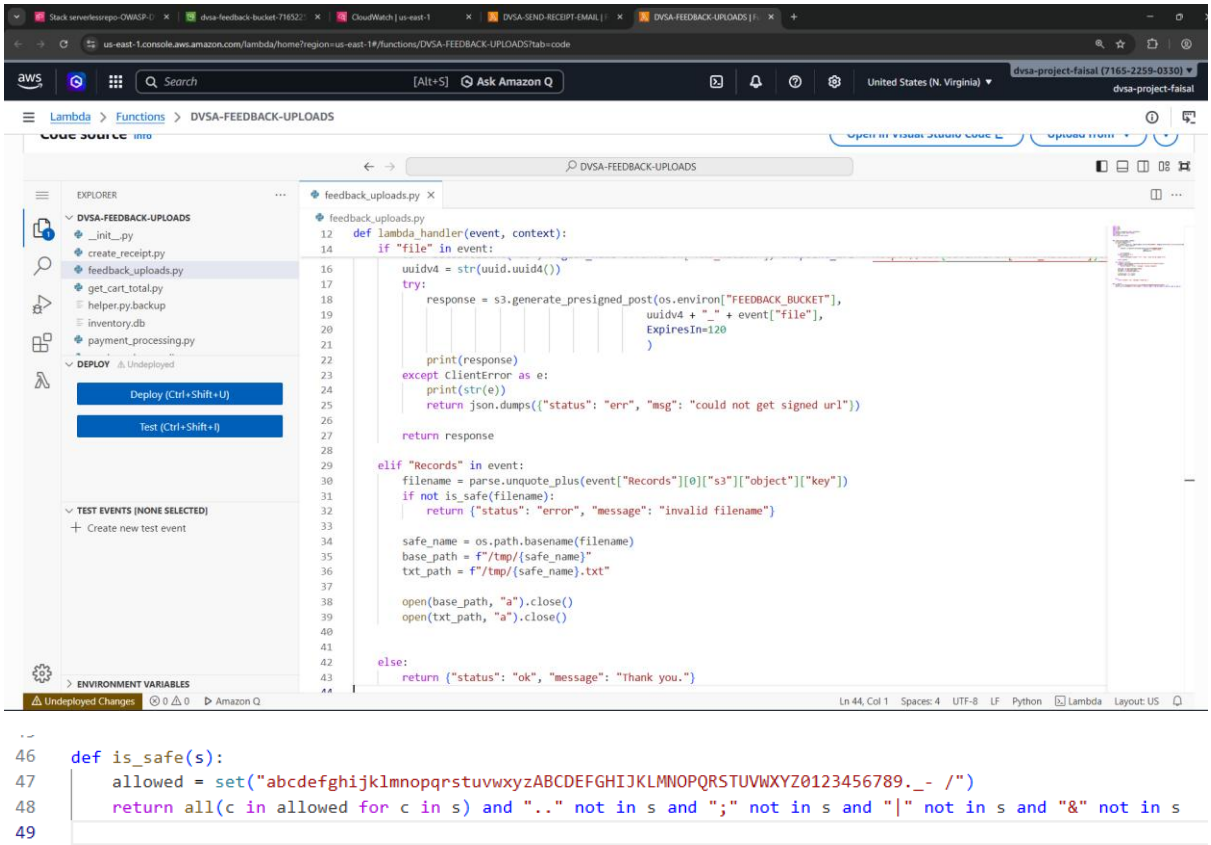


Figure 7: S3 feedback bucket configuration after remediation, showing that Block Public Access is enabled and the public access statement has been removed




```
def is_safe(s):
    allowed = set("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789._- /")
    return all(c in allowed for c in s) and ".." not in s and ";" not in s and "|" not in s and "&" not in s
```

Figure 8: Updated feedback_uploads.py code after remediation, showing the removal of unsafe shell execution and the addition of stricter filename validation.

Part 8) Verification After Fix

After applying the S3 configuration fix, I repeated the relevant checks on the feedback bucket. The bucket permissions confirmed that **Block Public Access** was enabled, and the previous public policy statement with Principal: "*" had been removed. Only the administrative statement for the AWS account remained.

I then tested the bucket again from the AWS Console using my authorized AWS account. The upload of a harmless file (lesson4-after-fix.txt) still succeeded and the object appeared in the bucket, confirming that legitimate administrative functionality still worked.

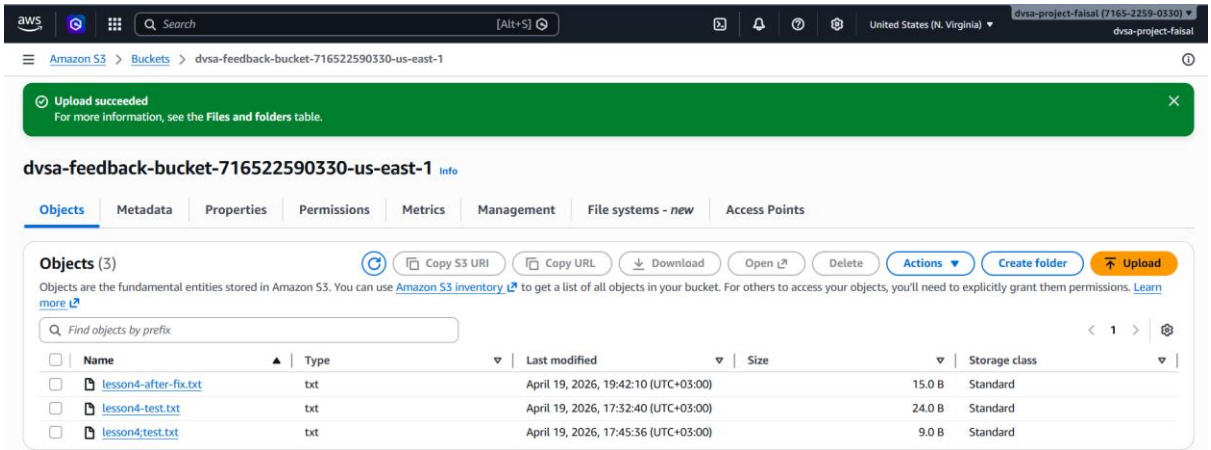


Figure 9: Post-fix verification showing that an authorized upload from the AWS Console still succeeds for normal administrative use.

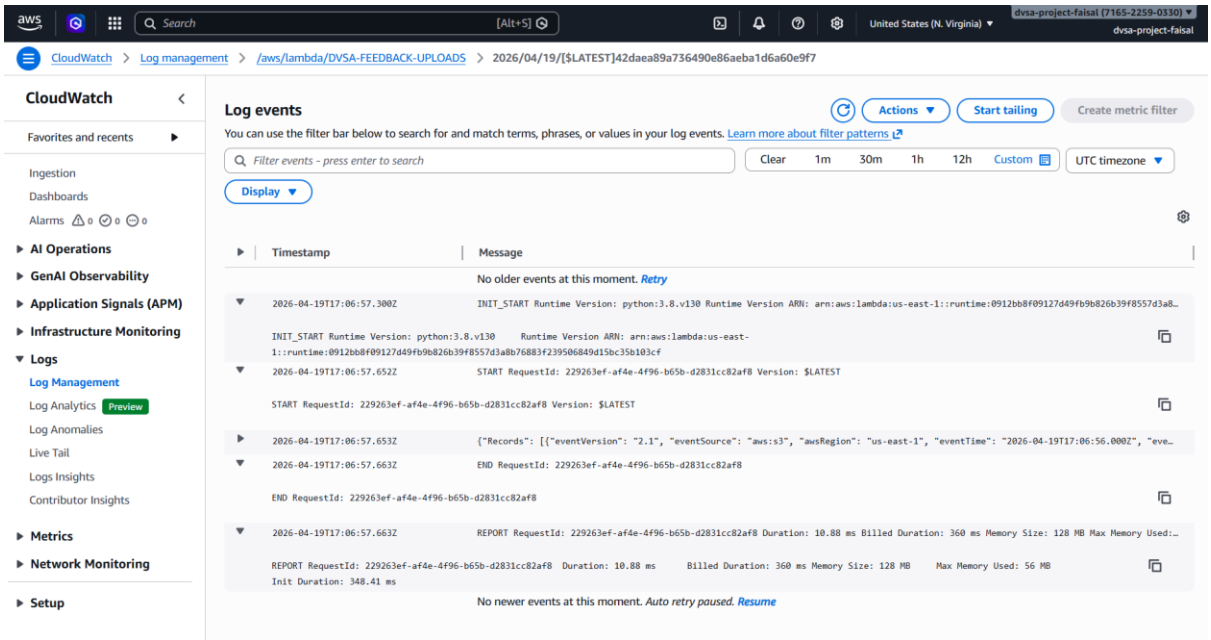


Figure 10: CloudWatch Logs after Lambda remediation showing that a special filename still triggers the Lambda event, but the previous shell-command errors no longer occur.

I also verified the backend remediation in the Lambda function DVSA-FEEDBACK-UPLOADS. After deploying the updated code, I uploaded another file with a special filename. CloudWatch Logs showed that the Lambda was still invoked for the S3 event and the object key was recorded, but the previous shell-related errors no longer appeared. This confirmed that attacker-controlled filenames were no longer reaching unsafe shell command execution.

The important change was that the bucket was no longer publicly exposed for anonymous upload, read, or delete operations. Therefore, the insecure cloud configuration on the S3 side was corrected without breaking intended authorized access.

This verification shows that the fix removed the public attack surface while preserving normal management access for the bucket owner.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Insecure Cloud Configuration	Only authorized users should be able to upload feedback files to the S3 feedback bucket, and uploaded filenames must not be able to influence unsafe backend processing.	S3 bucket permissions, bucket policy, event notification settings, Lambda code (feedback_uploads.py), uploaded object list, CloudWatch logs.	The feedback upload workflow is intended to accept controlled uploads and process them through DVSA-FEEDBACK-UPLOADS as part of the application workflow.	The bucket had Block Public Access disabled and a bucket policy with Principal: "*". I successfully uploaded files directly into the bucket, and CloudWatch showed that the uploads triggered Lambda execution. A special filename caused shell-related errors in the logs.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Insecure Cloud Configuration	The system allowed publicly writable storage to trigger backend processing on attacker-controlled object names. This violated the intended rule that	Accidental misconfiguration / security-relevant abuse	Enabled Block Public Access on dvsa-feedback-bucket-716522590330-us-east-1, removed the public bucket policy statement PublicWritefeedbackStatement, replaced os.system(...) in feedback_uploads.py with safe	After the fix, Block Public Access remained enabled, the public statement stayed removed, authorized AWS Console uploads still worked	Optional

	only authorized uploads should reach the backend and that filenames should not influence unsafe command execution.		file handling, and strengthened is_safe() validation.	normally, and a special filename no longer produced shell-command errors in CloudWatch Logs.	

Part 10) Takeaway / Lessons Learned

This lesson shows that insecure storage configuration in a serverless application can become a direct entry point into backend execution. A public S3 bucket is not only a storage problem; when object creation events trigger Lambda functions, it becomes part of the attack surface. The main secure design lessons are to apply least privilege to cloud storage, remove unnecessary public access, and never pass attacker-controlled filenames into unsafe shell commands.